

## Task 8 - Dyab Asdi - DA32435

```
In [63]: import pandas as pd
from mlxtend.frequent_patterns import apriori, association_rules, fpgrowth
```

```
In [64]: df_metadata = pd.read_csv('movies_metadata_clean.csv', encoding='iso-8859-1')
df_ratings = pd.read_csv('movies_ratings_small.csv', encoding='iso-8859-1')

# drop unimportant data
df_metadata = df_metadata.drop(["homepage", "imdb_id", "belongs_to_collection", "tagline"])
df_metadata.dropna(axis=0, inplace=True)
df_ratings.drop('timestamp', axis = 1)

# keep high ratings
df_ratings_high = df_ratings[df_ratings["rating"] >= 3.5]
```

```
In [65]: # create pivot table
pivot = df_ratings_high.pivot(index="userId", columns="movieId", values="rating")
pivot = pivot.notna().astype(int)
pivot
```

```
Out[65]: movieId  1  2  3  4  5  6  7  8  9  10  ...  160438  160563  160567  160590  160656  160718
userId
1      0  0  0  0  0  0  0  0  0  0  ...      0      0      0      0      0      0
2      0  0  0  0  0  0  0  0  0  0  1  ...      0      0      0      0      0      0
3      0  0  0  0  0  0  0  0  0  0  0  ...      0      0      0      0      0      0
4      0  0  0  0  0  0  0  0  0  0  1  ...      0      0      0      0      0      0
5      0  0  1  0  0  0  0  0  0  0  0  ...      0      0      0      0      0      0
...    ...  ...  ...  ...  ...  ...  ...  ...  ...  ...  ...  ...      ...      ...      ...      ...      ...      ...
667    0  0  0  0  0  0  1  0  0  0  0  ...      0      0      0      0      0      0
668    0  0  0  0  0  0  0  0  0  0  0  ...      0      0      0      0      0      0
669    0  0  0  0  0  0  0  0  0  0  0  ...      0      0      0      0      0      0
670    1  0  0  0  0  0  0  0  0  0  0  ...      0      0      0      0      0      0
671    1  0  0  0  0  0  0  0  0  0  0  ...      0      0      0      0      0      0
```

671 rows × 6993 columns



```
In [66]: frequent_movies1 = apriori(pivot, min_support=0.07, use_colnames=True)
frequent_movies2 = fpgrowth(pivot, min_support=0.07, use_colnames=True)

rules1 = association_rules(frequent_movies1, metric="lift", min_threshold=1.2)
rules2 = association_rules(frequent_movies2, metric="lift", min_threshold=1.2)
```

```
c:\Users\dyaba\Desktop\School\Spring 2025\Data Science\.venv\lib\site-packages\mlxtend\fre
quent_patterns\fpcommon.py:161: DeprecationWarning: DataFrames with non-bool types result
in worse computational performance and their support might be discontinued in the future.Pl
ease use a DataFrame with bool type
  warnings.warn(
c:\Users\dyaba\Desktop\School\Spring 2025\Data Science\.venv\lib\site-packages\mlxtend\fre
quent_patterns\fpcommon.py:161: DeprecationWarning: DataFrames with non-bool types result
in worse computational performance and their support might be discontinued in the future.Pl
ease use a DataFrame with bool type
  warnings.warn(
```

```
In [67]: rules1.sort_values(by=['lift'], ascending=False, inplace=True)
rules1.head()
```

Out[67]:

|               | antecedents                         | consequents                         | antecedent<br>support | consequent<br>support | support  | confidence | lift     | repr |
|---------------|-------------------------------------|-------------------------------------|-----------------------|-----------------------|----------|------------|----------|------|
| <b>128274</b> | (5952, 4993,<br>260, 1291,<br>1196) | (7153, 1210,<br>1198)               | 0.083458              | 0.092399              | 0.071535 | 0.857143   | 9.276498 |      |
| <b>126193</b> | (7153, 1210,<br>1198)               | (5952, 1196,<br>1291, 260)          | 0.092399              | 0.083458              | 0.071535 | 0.774194   | 9.276498 |      |
| <b>126124</b> | (5952, 1196,<br>1291, 260)          | (7153, 1210,<br>1198)               | 0.083458              | 0.092399              | 0.071535 | 0.857143   | 9.276498 |      |
| <b>128455</b> | (7153, 1210,<br>1198)               | (5952, 4993,<br>260, 1291,<br>1196) | 0.092399              | 0.083458              | 0.071535 | 0.774194   | 9.276498 |      |
| <b>128384</b> | (4993, 1210,<br>7153, 1198)         | (5952, 1196,<br>1291, 260)          | 0.092399              | 0.083458              | 0.071535 | 0.774194   | 9.276498 |      |



```
In [68]: rules2.sort_values(by=['confidence'], ascending=False, inplace=True)
rules2.head()
```

Out[68]:

|        | antecedents                    | consequents | antecedent support | consequent support | support  | confidence | lift     | repr |
|--------|--------------------------------|-------------|--------------------|--------------------|----------|------------|----------|------|
| 114505 | (5952, 3578, 1210, 1196)       | (4993)      | 0.071535           | 0.251863           | 0.071535 | 1.0        | 3.970414 |      |
| 114475 | (5952, 3578, 1210, 260)        | (4993)      | 0.073025           | 0.251863           | 0.073025 | 1.0        | 3.970414 |      |
| 114566 | (5952, 260, 1196, 3578, 1210)  | (4993)      | 0.070045           | 0.251863           | 0.070045 | 1.0        | 3.970414 |      |
| 59446  | (5952, 1291, 1198, 7153, 1270) | (4993)      | 0.070045           | 0.251863           | 0.070045 | 1.0        | 3.970414 |      |
| 76482  | (5952, 260, 2571, 1196, 1210)  | (4993)      | 0.096870           | 0.251863           | 0.096870 | 1.0        | 3.970414 |      |

In [ ]: